

The `filter()` function returns a filter object, which we can once again pass to the `list()` function. Only the even integers remain in the filtered list.

But the `map()` and `filter()` functions are outdated ways to create mapped or filtered lists in Python. Instead, you can now create them with list comprehensions. List comprehensions not only free you from writing out a lambda function, but are also faster than `map()` and `filter()`.

Here we replicate the `map()` function example using a list comprehension:

```
>>> [str(n) for n in [8, 16, 18, 19, 12, 1, 6, 7]]
['8', '16', '18', '19', '12', '1', '6', '7']
```

Notice that the `str(n)` part of the list comprehension is similar to `lambda n: str(n)`.

And here we replicate the `filter()` function example using a list comprehension:

```
>>> [n for n in [8, 16, 18, 19, 12, 1, 6, 7] if n % 2 == 0]
[8, 16, 18, 12, 6]
```

Notice that the `if n % 2 == 0` part of the list comprehension is similar to `lambda n: n % 2 == 0`.

Many languages have a concept of functions as first-class objects, allowing for the existence of higher-order functions, including mapping and filtering functions.

Return Values Should Always Have the Same Data Type

Python is a dynamically typed language, which means that Python functions and methods are free to return values of any data type. But to make your functions more predictable, you should strive to have them return values of only a single data type.

For example, here's a function that, depending on a random number, returns either an integer value or a string value: